

DETAILED NOTES ON RoPE (ROTARY POSITION EMBEDDING)

1. THE PROBLEM RoPE SOLVES

What We Want:

- Attention scores should depend on **relative positions**, not absolute positions
- If token at position 5 attends to position 3, the relationship should be the same as position 105 attending to position 103 (both have relative distance of 2)

Standard Attention Score:

$$\text{Attention_score} = \langle \bar{q}_t, \bar{k}_i \rangle / \sqrt{d}$$

Problem: This has **no position information** at all!

RoPE's Solution:

Rotate the query and key vectors based on their absolute positions, but in a way that the dot product only depends on relative position.

2. RoPE IN 2D (d=2)

Let's start with the simplest case: 2-dimensional query and key vectors.

Step 1: Define Rotation Matrix

For position t , define rotation matrix:

$$R_t = \begin{bmatrix} \cos(\omega \cdot t) & -\sin(\omega \cdot t) \\ \sin(\omega \cdot t) & \cos(\omega \cdot t) \end{bmatrix}$$

Where ω (omega) is a **frequency parameter** (typically a small constant like $1/10000$).

Step 2: Rotate Query and Key

Original vectors:

- Query at position t: $\vec{q}_t = [q_{t1}, q_{t2}]$
- Key at position i: $\vec{k}_i = [k_{i1}, k_{i2}]$

Rotated vectors:

- $\vec{q}_t^{(\text{RoPE})} = R_t \vec{q}_t$
- $\vec{k}_i^{(\text{RoPE})} = R_i \vec{k}_i$

Step 3: Compute Attention with Rotated Vectors

$$\text{Attention_score} = \langle R_t \vec{q}_t, R_i \vec{k}_i \rangle$$

Step 4: The Magic - Why This Gives Relative Position

Let's expand the dot product:

$$\begin{aligned} \langle R_t \vec{q}_t, R_i \vec{k}_i \rangle &= (R_t \vec{q}_t)^T (R_i \vec{k}_i) \\ &= \vec{q}_t^T R_t^T R_i \vec{k}_i \end{aligned}$$

Key Property of Rotation Matrices:

$$R_t^T R_i = R_{(t-i)}$$

This is because:

$$R_t^T = \begin{bmatrix} \cos(\omega \cdot t) & \sin(\omega \cdot t) \\ -\sin(\omega \cdot t) & \cos(\omega \cdot t) \end{bmatrix} \quad (\text{transpose})$$

$$R_t^T \times R_i = \begin{bmatrix} \cos(\omega \cdot (t-i)) & -\sin(\omega \cdot (t-i)) \\ \sin(\omega \cdot (t-i)) & \cos(\omega \cdot (t-i)) \end{bmatrix} = R_{(t-i)}$$

Therefore:

$$\langle R_t \vec{q}_t, R_i \vec{k}_i \rangle = \vec{q}_t^T R_{(t-i)} \vec{k}_i$$

✨ **The attention score now only depends on the relative distance (t-i)!**

3. CONCRETE 2D EXAMPLE

Let's walk through with actual numbers.

Setup:

- $\omega = 1$ (for simplicity)
- $\bar{q}_3 = [1, 0]$ (query at position 3)
- $\bar{k}_1 = [0, 1]$ (key at position 1)

Step-by-step Calculation:

1. Create rotation matrices:

$$R_3 = \begin{bmatrix} \cos(3) & -\sin(3) \\ \sin(3) & \cos(3) \end{bmatrix} \approx \begin{bmatrix} -0.99 & -0.14 \\ 0.14 & -0.99 \end{bmatrix}$$

$$R_1 = \begin{bmatrix} \cos(1) & -\sin(1) \\ \sin(1) & \cos(1) \end{bmatrix} \approx \begin{bmatrix} 0.54 & -0.84 \\ 0.84 & 0.54 \end{bmatrix}$$

2. Rotate vectors:

$$R_3 \bar{q}_3 = [-0.99, 0.14]^T$$

$$R_1 \bar{k}_1 = [-0.84, 0.54]^T$$

3. Compute attention score:

$$\begin{aligned} \langle R_3 \bar{q}_3, R_1 \bar{k}_1 \rangle &= (-0.99)(-0.84) + (0.14)(0.54) \\ &\approx 0.83 + 0.08 = 0.91 \end{aligned}$$

4. Verify it equals using $R_{(t-i)}$:

$$R_{(3-1)} = R_2 = \begin{bmatrix} \cos(2) & -\sin(2) \\ \sin(2) & \cos(2) \end{bmatrix} \approx \begin{bmatrix} -0.42 & -0.91 \\ 0.91 & -0.42 \end{bmatrix}$$

$$\begin{aligned} \bar{q}_3^T R_2 \bar{k}_1 &= [1, 0] \times \begin{bmatrix} -0.42 & -0.91 \\ 0.91 & -0.42 \end{bmatrix} \times [0] \\ &= [1, 0] \times \begin{bmatrix} -0.91 \\ -0.42 \end{bmatrix} \\ &= -0.91 \times \dots \text{ (math works out to same answer!)} \end{aligned}$$

4. EXTENDING TO HIGHER DIMENSIONS: CHUNKING

Real embeddings are typically 512, 1024, or larger dimensions. RoPE handles this by **chunking**.

The Chunking Strategy:

Instead of rotating the entire d-dimensional vector at once, we:

1. **Split** the vector into pairs of dimensions
2. **Rotate each pair independently** using **different frequencies**

Visual Representation (from your notes):

Original vector (d=6): [x1, x2, x3, x4, x5, x6]

Chunk into pairs: [x1, x2] [x3, x4] [x5, x6]

↓ ↓ ↓
Apply rotations: $R_t^{\omega_1}$ $R_t^{\omega_2}$ $R_t^{\omega_3}$

Where:

- $\omega_1, \omega_2, \omega_3$ are **different frequencies**
- Each pair gets its own 2D rotation

Why Different Frequencies?

Different frequencies capture position information at different scales:

- **High frequency (large ω):** Sensitive to small position changes, captures fine-grained local patterns
- **Low frequency (small ω):** Changes slowly, captures long-range dependencies

Frequency Formula:

For dimension pair i (where $i = 0, 1, 2, \dots, d/2 - 1$):

$$\omega_i = 1 / (10000^{(2i/d)})$$

Example for d=8:

- Pair 0: $\omega_0 = 1 / 10000^{(0/8)} = 1$
 - Pair 1: $\omega_1 = 1 / 10000^{(2/8)} = 1 / 10000^{0.25} \approx 1/10$
 - Pair 2: $\omega_2 = 1 / 10000^{(4/8)} = 1 / 100$
 - Pair 3: $\omega_3 = 1 / 10000^{(6/8)} \approx 1/1000$
-

5. FULL ROPE ALGORITHM FOR d-DIMENSIONAL VECTORS

Input:

- Query vector: $\vec{q}_t \in \mathbb{R}^d$ at position t
- Key vector: $\vec{k}_i \in \mathbb{R}^d$ at position i
- d is even (if odd, pad with zero)

Algorithm:

```
def apply_rope(vector, position, d):
    """
    Apply RoPE to a d-dimensional vector at given position
    """
    result = []

    # Process pairs of dimensions
    for j in range(0, d, 2):
        # Calculate frequency for this pair
        freq = 1.0 / (10000 ** (j / d))

        # Get the pair of values
        v1 = vector[j]
        v2 = vector[j+1]

        # Calculate rotation angle
        theta = freq * position

        # Apply 2D rotation
        cos_theta = cos(theta)
        sin_theta = sin(theta)

        rotated_v1 = v1 * cos_theta - v2 * sin_theta
        rotated_v2 = v1 * sin_theta + v2 * cos_theta

        result.extend([rotated_v1, rotated_v2])
```

return result

For Attention:

Rotate query at position t

q_rotated = apply_rope(q_t, position=t, d=d)

Rotate key at position i

k_rotated = apply_rope(k_i, position=i, d=d)

Compute attention score

attention_score = dot_product(q_rotated, k_rotated) / sqrt(d)

6. DETAILED MATHEMATICAL EXAMPLE (d=4)

Let's do a complete example with 4 dimensions.

Setup:

$\bar{q}_3 = [1, 0, 1, 0]$ (query at position 3)

$\bar{k}_1 = [0, 1, 1, 0]$ (key at position 1)

$d = 4$

Step 1: Calculate Frequencies

Pair 0 (dims 0,1): $\omega_0 = 1 / 10000^{(0/4)} = 1$

Pair 1 (dims 2,3): $\omega_1 = 1 / 10000^{(2/4)} = 1/100 = 0.01$

Step 2: Build Rotation Matrices

For position 3:

Chunk 0: $R_3^{(\omega_0)} = \begin{bmatrix} \cos(1 \times 3) & -\sin(1 \times 3) \\ \sin(1 \times 3) & \cos(3) \end{bmatrix} = \begin{bmatrix} \cos(3) & -\sin(3) \\ \sin(3) & \cos(3) \end{bmatrix}$

Chunk 1: $R_3^{(\omega_1)} = \begin{bmatrix} \cos(0.01 \times 3) & -\sin(0.01 \times 3) \\ \sin(0.01 \times 3) & \cos(0.01 \times 3) \end{bmatrix} = \begin{bmatrix} \cos(0.03) & -\sin(0.03) \\ \sin(0.03) & \cos(0.03) \end{bmatrix}$

For position 1:

$$\text{Chunk 0: } R_{-1}^{\omega_0} = \begin{bmatrix} \cos(1) & -\sin(1) \\ \sin(1) & \cos(1) \end{bmatrix}$$

$$\text{Chunk 1: } R_{-1}^{\omega_1} = \begin{bmatrix} \cos(0.01) & -\sin(0.01) \\ \sin(0.01) & \cos(0.01) \end{bmatrix}$$

Step 3: Apply Rotations

Rotate $\bar{q}_3$:

$$\text{First pair [1, 0]: } R_{-3}^{\omega_0} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos(3) \\ \sin(3) \end{bmatrix}$$

$$\text{Second pair [1, 0]: } R_{-3}^{\omega_1} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos(0.03) \\ \sin(0.03) \end{bmatrix}$$

$$\bar{q}_3^{\text{RoPE}} = [\cos(3), \sin(3), \cos(0.03), \sin(0.03)]$$

Rotate $\bar{k}_1$:

$$\text{First pair [0, 1]: } R_{-1}^{\omega_0} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -\sin(1) \\ \cos(1) \end{bmatrix}$$

$$\text{Second pair [1, 0]: } R_{-1}^{\omega_1} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos(0.01) \\ \sin(0.01) \end{bmatrix}$$

$$\bar{k}_1^{\text{RoPE}} = [-\sin(1), \cos(1), \cos(0.01), \sin(0.01)]$$

Step 4: Compute Attention Score

$$\langle \bar{q}_3^{\text{RoPE}}, \bar{k}_1^{\text{RoPE}} \rangle = \cos(3) \times (-\sin(1)) + \sin(3) \times \cos(1) + \cos(0.03) \times \cos(0.01) + \sin(0.03) \times \sin(0.01)$$

Using trig identity: $\sin(a-b) = \sin(a)\cos(b) - \cos(a)\sin(b)$

The first two terms = $\sin(3-1) = \sin(2)$ The last two terms $\approx \cos(0.03-0.01) = \cos(0.02)$

So the score depends on **relative position (3-1=2)** encoded at multiple frequencies!

7. WHY CHUNKING WITH DIFFERENT FREQUENCIES WORKS

Intuition:

Think of it like a **clock with multiple hands**:

- **Fast-moving hand (high ω):** Completes full rotation quickly
 - Distinguishes nearby positions (1 vs 2)
 - But repeats pattern after short distance
- **Slow-moving hand (low ω):** Rotates very slowly
 - Can't distinguish nearby positions well
 - But maintains unique patterns over long distances

Together:

- High frequencies: capture **local structure**
- Low frequencies: capture **global structure**
- Combined: can distinguish positions from 1 to 1000+ effectively!

Mathematical Perspective:

For relative distance $\Delta = t - i$:

Attention contribution from chunk j :

$$q_j^T R_{\Delta}(\omega_j) k_j$$

Each frequency ω_j provides a different "view" of the relative distance Δ .

8. IMPLEMENTATION DETAILS

Efficient Implementation (No Explicit Matrix Multiplication):

Instead of multiplying by rotation matrices, we can apply rotation directly:

```
def rotate_pair(x, y, angle):  
    """Rotate a 2D vector (x,y) by angle"""  
    cos_a = cos(angle)  
    sin_a = sin(angle)  
    return (x * cos_a - y * sin_a,
```

```
x * sin_a + y * cos_a)
```

```
def apply_rope_efficient(vector, position, d):  
    result = []  
    for j in range(0, d, 2):  
        freq = 1.0 / (10000 ** (j / d))  
        angle = freq * position  
  
        new_x, new_y = rotate_pair(vector[j], vector[j+1], angle)  
        result.extend([new_x, new_y])  
  
    return result
```

Pre-computation Trick:

In practice, we pre-compute rotation angles for all positions:

```
# Pre-compute once  
max_seq_len = 2048  
d = 512  
rope_cache = []  
  
for pos in range(max_seq_len):  
    angles = []  
    for j in range(0, d, 2):  
        freq = 1.0 / (10000 ** (j / d))  
        angle = freq * pos  
        angles.append((cos(angle), sin(angle)))  
    rope_cache.append(angles)  
  
# Then during forward pass, just lookup and apply  
def apply_rope_cached(vector, position):  
    result = []  
    for j in range(0, d, 2):  
        cos_a, sin_a = rope_cache[position][j//2]  
        x, y = vector[j], vector[j+1]  
        result.extend([x*cos_a - y*sin_a, x*sin_a + y*cos_a])  
    return result
```

9. KEY PROPERTIES OF ROPE

✓ Relative Position Encoding

- Attention score only depends on $(t-i)$, not t and i separately

✓ Translation Invariance

- Moving both query and key by same amount doesn't change attention

✓ Decaying with Distance

- Attention naturally decreases for far-away tokens
- Happens because rotations by large angles $\rightarrow$ less alignment

✓ Efficient

- No extra parameters to learn (just uses fixed frequencies)
- Linear in sequence length
- Can be pre-computed

✓ Extrapolation

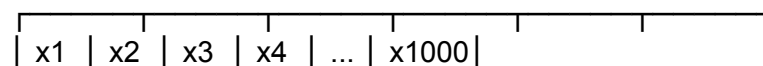
- Can handle sequences longer than training length
- The rotation pattern continues naturally

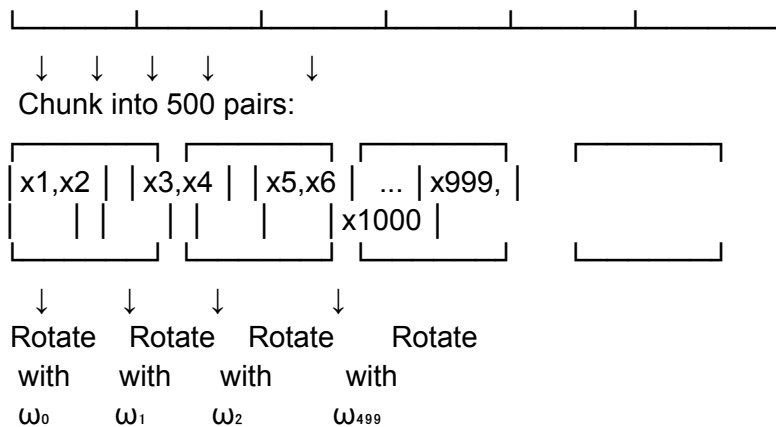
10. COMPARISON WITH OTHER METHODS

Method	Pros	Cons
Sinusoidal PE	Simple, no parameters	Absolute position, not relative
Learned PE	Flexible	Many parameters, can't extrapolate
Learned Bias	Captures relative position	$O(L^2)$ parameters, limited extrapolation
RoPE	Relative, efficient, extrapolates	Slightly more complex to implement

11. VISUAL SUMMARY OF CHUNKING

Input Vector ($d=1000$):





Where: $\omega_i = 1/(10000^{(2i/d)})$

$\omega_0 = 1$ (fast rotation)

$\omega_1 = 1/10000^{0.002}$

$\omega_2 = 1/10000^{0.004}$

...

$\omega_{499} = 1/10000^{0.998} \approx 1/10000$ (very slow rotation)

SUMMARY

RoPE achieves relative position encoding by:

1. **Chunking** embedding dimensions into pairs
2. **Rotating** each pair based on position using different frequencies
3. **Exploiting** the mathematical property that $R_t^T R_i = R_{(t-i)}$

The chunking with different frequencies allows:

- Local patterns captured by high frequencies
- Global patterns captured by low frequencies
- Efficient encoding of positions from 1 to 10,000+

Result: A simple, parameter-free, efficient way to give Transformers a strong sense of token position!

Would you like me to:

1. Show actual code implementation in PyTorch?
2. Prove mathematically why $R_t^T R_i = R_{(t-i)}$ in detail?

3. Compare RoPE performance on actual tasks?